

Nama/ NIM : Ihsan Surya Jamil / 224260023
Kelas/ Prodi : A2/ Sistem Informasi
Semester : Semester 7
Matkul : Analisis & Perancangan Sistem Informasi
Dosen : Pa Aan Ansen, ST.,M.Kom

PERTANYAAN

- Apa yang dimaksud USE CASE Realization dalam konteks perancangan system berbasis object
- Jelaskan tahapan yang dilakukan Ketika melakukan Realization Use Case dari sebuah skenario pengguna
- Mengapa Use Case Realization penting dalam pengembangan perangkat lunak berbasis object

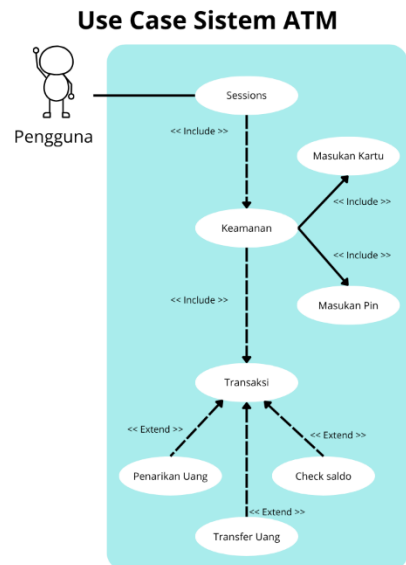
JAWABAN

- Use Case Realization (Realisasi Use Case) adalah proses untuk menjembatani kesenjangan antara pandangan analisis (kebutuhan fungsional yang digambarkan oleh Use Case) dan pandangan desain (bagaimana objek-objek dalam sistem berkolaborasi untuk memenuhi kebutuhan tersebut).

- Use Case:

Realisasinya:

- Identifikasi Objek/Kelas yang Terlibat:
 - Baca skenario langkah demi langkah dan identifikasi kata benda. Ini adalah kandidat untuk menjadi objek.
 - Biasanya, objek-objek ini dikategorikan menjadi tiga jenis (pola BCE):
 - Boundary: Objek yang berinteraksi langsung dengan Aktor (misal: FormLogin, LayarMenu, TombolATM).
 - Control: Objek yang mengatur logika bisnis atau alur kerja. Ia bertindak sebagai "manajer" untuk sebuah use case (misal: ManajerTransaksi, KontrolPencarian).
 - Entity: Objek yang menyimpan data dan biasanya bersifat permanen (misal: Akun, Buku, Anggota).



2. Petakan Alur Skenario ke Diagram Interaksi:

- Gunakan Sequence Diagram (Diagram Sekuens) sebagai "kanvas" Anda.
- Letakkan Aktor dan objek-objek (Boundary, Control, Entity) yang telah Anda identifikasi di bagian atas.
- Ikuti setiap langkah dalam skenario (Main Flow):
 - Langkah 1: "Anggota mencari buku."
 - *Realisasi*: Aktor Anggota mengirim pesan cari(keyword) ke objek LayarPencarian (Boundary).
 - Langkah 2: "Sistem menampilkan hasil pencarian."
 - *Realisasi*: LayarPencarian mengirim pesan prosesCari(keyword) ke KontrolPencarian (Control). KontrolPencarian mencari ke objek DatabaseBuku (Entity) dan mengembalikan hasilnya ke LayarPencarian untuk ditampilkan.

3. Tentukan Tanggung Jawab (Metode/Fungsi):

- Setiap "pesan" (panah) yang dikirim ke sebuah objek dalam Sequence Diagram menjadi sebuah *responsibility* atau metode (fungsi) yang harus dimiliki oleh kelas dari objek tersebut.
- Contoh: Karena KontrolPencarian menerima pesan prosesCari(keyword), maka Kelas KontrolPencarian harus memiliki metode public void prosesCari(String keyword).

4. Perbarui Diagram Kelas (Class Diagram):

- Setelah semua metode dari skenario ditemukan, perbarui Diagram Kelas Anda untuk menambahkan metode-metode baru ini ke kelas yang sesuai. Ini melengkapi pandangan desain statis Anda.

C. Use Case Realization sangat penting dalam pengembangan berorientasi objek karena beberapa alasan utama:

- Menemukan Objek dan Kelas: Ini adalah cara paling sistematis untuk menemukan kelas-kelas yang Anda butuhkan dalam desain Anda, bukan hanya "menebak" berdasarkan pengalaman.
- Menentukan Tanggung Jawab (Metode): Ini adalah mekanisme utama untuk menentukan metode (fungsi) apa yang harus dimiliki oleh setiap kelas dan siapa yang memanggilmnya. Tanpa ini, sulit untuk menentukan logika di dalam kelas.
- Menerapkan Prinsip OO: Proses ini secara alami mendorong prinsip-prinsip object-oriented yang baik, seperti Enkapsulasi (menyembunyikan detail) dan Pemisahan Tanggung Jawab (Separation of Concerns). Objek Boundary hanya mengurus tampilan, Entity hanya mengurus data, dan Control hanya mengurus logika.
- Ketertelusuran (Traceability): Ini menciptakan jejak yang jelas. Jika ada perubahan pada kebutuhan (Use Case), Anda tahu persis harus melihat *Sequence Diagram* yang mana dan mengubah *metode* di *kelas* yang mana.
- Memvalidasi Desain: Ini membantu Anda memvalidasi arsitektur Anda. Jika Sequence Diagram Anda terlihat sangat rumit (terlalu banyak pesan bolak-balik), itu pertanda

bahwa desain Anda mungkin perlu diperbaiki (misalnya, kelas Anda terlalu banyak tanggung jawab).